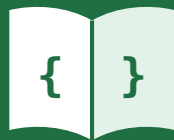

Module 12 — Security with Spring Security

Lesson Notes

Spring Boot Edu



2026-09-24

Contents

1. Learning Goals	2
2. Concepts	2
2.1 Authentication and Authorization	2
2.2 The Filter Chain	2
2.3 Sessions, CSRF and CORS	3
3. Step-by-Step Examples	3
3.1 The Browser Chain	4
3.2 Users and Passwords	5
3.3 The API Chain	6
3.4 Method Security	7
3.5 A JWT Resource Server	8
3.6 Our Own Authorization Server	8
3.7 OAuth2 Login (“Log in with GitHub”)	10
3.8 CORS	10
3.9 Testing	11
4. Common Mistakes and Best Practices	11
5. Summary	12
6. Further Reading	12

1. Learning Goals

By the end of this module you will be able to:

- Explain how the Spring Security filter chain decides about every request
- Use several `SecurityFilterChains`: a stateless one for the API, a session-based one for the browser
- Store users in PostgreSQL and passwords as salted BCrypt hashes
- Choose between form login, HTTP Basic and bearer tokens, and know when CSRF and CORS matter
- Protect service methods with `@PreAuthorize` and `@PostAuthorize`
- Issue JWTs with Spring Authorization Server and accept them as a resource server
- Offer “Log in with GitHub” (OAuth2 login)
- Test all of this with `spring-security-test`

Prerequisites: Modules 03 (REST) and 06 (PostgreSQL) · **Estimated time:** 6 hours · **Docker required**

2. Concepts

2.1 Authentication and Authorization

Authentication	Authorization
Who are you?	What may you do?
Password, token, certificate	Roles, scopes, ownership rules
Fails with 401 Unauthorized (“log in first”)	Fails with 403 Forbidden (“you are known, but not allowed”)

The result of authentication is an `Authentication` object in the `SecurityContext`: a name, and a list of **authorities** such as `ROLE_ADMIN` or `SCOPE_books.write`. `hasRole('ADMIN')` is the same check as `hasAuthority('ROLE_ADMIN')`.

2.2 The Filter Chain

Spring Security is a chain of servlet filters that runs **before** your controller. Each `SecurityFilterChain` bean has a matcher, and the first chain whose matcher fits handles the request:

```
request → chain 1: /oauth2/**, /.well-known/** (Authorization Server)
        → chain 2: /api/** (stateless: Basic or JWT)
        → chain 3: everything else (browser: form login, session, CSRF)
```

Inside a chain, the filters authenticate the request (form, Basic, bearer token...), and the last one, `AuthorizationFilter`, checks the rules.

2.3 Sessions, CSRF and CORS

- A **session** cookie is sent by the browser automatically with every request to your site, also when another site triggers the request. That is what a **CSRF** attack abuses. CSRF protection therefore matters for cookie-based (browser) applications, and Spring Security enables it by default.
- A **stateless** API that expects an explicit Authorization header on every request has no cookie to abuse.
- **CORS** is about the opposite direction: whether JavaScript from another origin may read your API's responses. The browser asks first with a preflight `OPTIONS` request.

3. Step-by-Step Examples

With Docker running, start the application. PostgreSQL starts from the root `compose.yaml`:

```
./mvnw -pl modules/12-security/lesson spring-boot:run
```

`spring-boot:run` activates the **dev** profile. Only this profile loads the demo users and the demo client secret:

```
spring:
  flyway:
    locations: classpath:db/migration,classpath:db/demo # + demo users with known
    ↪ passwords
  security:
    oauth2:
      authorizationserver:
        client:
          bookstore-cli:
            registration:
              client-secret: "{noop}dev-only-secret" # plain text: acceptable
    ↪ only on your machine
  bookstore:
    tour:
```

```

    enabled: true                                # the tour uses the demo
    ↪ credentials

```

Without the dev profile, the demo users do not exist, and the application refuses to start unless the environment variable `BOOKSTORE_CLI_SECRET` holds the client secret (ideally as `{bcrypt}...`). Try it: `./mvnw -pl modules/12-security/lesson spring-boot:run -Dspring-boot.run.profiles=prod` stops with `Could not resolve placeholder 'BOOKSTORE_CLI_SECRET'.`

The tour (LessonTour, dev profile only) calls the running server over HTTP, like a real client:

```

== 3.3 Public and protected
GET /api/books (anonymous): [{"isbn":"9780134685991","title":"Effective Java"}, ...]
POST /api/books as ada (CUSTOMER): 403 Forbidden
== 3.6 A token from our Authorization Server
access_token: eyJraWQiOiI3MGE2MWFjMC00YzAxLTRhMWItYjAz...
== 3.5 Calling the API with the JWT
GET /api/me: {"name":"bookstore-
    ↪ cli","type":"JwtAuthenticationToken","authorities":["FACTOR_BEARER","SCOPE_books.read","SCOPE_books.read"]}

```

! Warning

Known demo passwords and `{noop}` secrets are acceptable only on your own machine. That is why they live in the dev profile and in the separate Flyway location `db/demo`, which a deployment never loads.

3.1 The Browser Chain

```

@Bean
@Order(3)
SecurityFilterChain webChain(HttpSecurity http,
    ↪ ObjectProvider<ClientRegistrationRepository> oauth2Clients)
    throws Exception {
    http.authorizeHttpRequests(authorize -> authorize
        .requestMatchers("/", "/error").permitAll()
        .anyRequest().authenticated())
        .formLogin(withDefaults())                // Spring Security generates
        ↪ /login
        .logout(withDefaults());                  // CSRF protection stays on
        ↪ (the default)
    if (oauth2Clients.getIfAvailable() != null) { // only if a provider is
        ↪ configured (lesson 3.7)
        http.oauth2Login(withDefaults());
    }
}

```

```

    }
    return http.build();
}

```

- / is public. Everything else needs a login. An anonymous request for /account is redirected to /login.
- formLogin() generates a login page and handles the POST /login. After a successful login, the user is sent back to the page they wanted, or to /. A wrong password leads to /login?error.
- CSRF protection stays switched on: a POST from the browser needs the hidden CSRF token that Spring Security puts into its forms. WebChainTest shows that the same POST without a token gets **403**.

3.2 Users and Passwords

The users live in PostgreSQL, in the tables JdbcUserDetailsManager expects:

```

-- The default schema of JdbcUserDetailsManager (see
↪ org/springframework/security/core/userdetails/jdbc/users.ddl)
CREATE TABLE users (
    username VARCHAR(50) NOT NULL PRIMARY KEY,
    password VARCHAR(500) NOT NULL,           -- "{bcrypt}$2a$10$...": the prefix names
    ↪ the algorithm
    enabled BOOLEAN NOT NULL
);

CREATE TABLE authorities (
    username VARCHAR(50) NOT NULL REFERENCES users (username),
    authority VARCHAR(50) NOT NULL           -- "ROLE_ADMIN" → hasRole('ADMIN')
);

CREATE UNIQUE INDEX ix_auth_username ON authorities (username, authority);

```

```

@Bean
UserDetailsManager users(dataSource) {
    return new JdbcUserDetailsManager(dataSource);    // tables "users" and
    ↪ "authorities" (Flyway V1)
}

@Bean
PasswordEncoder passwordEncoder() {
    return PasswordEncoderFactories.createDelegatingPasswordEncoder(); // bcrypt now,
    ↪ other ids still readable
}

```

- Passwords are stored as {bcrypt}\$2a\$10\$.... BCrypt adds a random **salt** (the same password gives a different hash every time) and is deliberately slow, so stolen hashes are expensive to crack.
- The prefix names the algorithm. The DelegatingPasswordEncoder encodes new passwords with BCrypt and can still check old hashes with another prefix, so you can change the algorithm without forcing everybody to reset their password.
- Form login and HTTP Basic both use this UserDetailsService bean.
- The schema comes from db/migration (all environments), the demo users from db/demo (dev profile only).

3.3 The API Chain

```
@Bean
@Order(2)
SecurityFilterChain apiChain(HttpSecurity http) throws Exception {
    http.securityMatcher("/api/**")
        .authorizeHttpRequests(authorize -> authorize
            .requestMatchers(HttpMethod.GET, "/api/books/**").permitAll()
            .requestMatchers(HttpMethod.POST, "/api/books/**")
                .hasAnyAuthority("ROLE_ADMIN", "SCOPE_books.write") // a person
                ↪ or a token
            .requestMatchers("/api/me", "/api/orders/**").authenticated()
            .anyRequest().denyAll() // deny what no rule
                ↪ allows
        .httpBasic(withDefaults()) // username +
            ↪ password (curl, scripts) ...
        .oauth2ResourceServer(resourceServer -> resourceServer.jwt(withDefaults()))
            ↪ // ... or a JWT
        .sessionManagement(session ->
            ↪ session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        // No session cookie. Careful: browsers cache Basic credentials and send them
        ↪ on their own -
        // for a browser front end use bearer tokens only, or keep CSRF on (lesson 3.3)
        .csrf(csrf -> csrf.disable())
        .cors(withDefaults()); // uses the
            ↪ CorsConfigurationSource bean
    return http.build();
}
```

- securityMatcher("/api/**"): this chain handles only API requests. It has a lower order number than the browser chain, so it is asked first.
- Reading books is public. Writing needs ROLE_ADMIN (a person) **or** SCOPE_books.write (a token).
- No credentials → **401** with a WWW-Authenticate header. A customer's valid credentials → **403**.

- STATELESS: no session is created, every request must authenticate itself. That is why CSRF can be switched off here.
- `anyRequest().denyAll()`: an API path that no rule mentions is refused, even for an admin. New endpoints stay closed until someone writes a rule for them.

! Warning

Switching CSRF off is safe for **bearer tokens**, because the browser never adds them on its own. HTTP Basic is different: once a user has typed Basic credentials into the browser's dialog, the browser sends them with later requests by itself. HTTP Basic in this chain is meant for curl and scripts. For a browser front end, accept only bearer tokens on the API, or keep CSRF protection on.

3.4 Method Security

URL rules cannot express “a customer may only see **their own** order”. Method security checks the rule where the data is:

```
@Service
public class OrderService {

    private final Map<Long, Order> orders = Map.of(
        1L, new Order(1, "ada", "9780134685991", 1),
        2L, new Order(2, "bob", "9781617297571", 2));

    @PreAuthorize("hasRole('ADMIN')") // checked before
    ⇨ the method runs
    public List<Order> findAll() {
        return List.copyOf(orders.values());
    }

    @PostAuthorize("returnObject.customer() == authentication.name or hasRole('ADMIN')")
    ⇨ // checked on the result
    public Order find(long id) {
        Order order = orders.get(id);
        if (order == null) {
            throw new NoSuchElementException("No order " + id);
        }
        return order;
    }
}
```

- `@PreAuthorize` runs before the method: `findAll` is for admins only.
- `@PostAuthorize` runs after it and can look at the result (`returnObject`): Ada gets order 1, but a request for Bob's order 2 ends with **403**.

- The rules apply to every caller of the service, not only to one URL. `OrderSecurityTest` calls `findAll()` directly and gets an `AuthorizationDeniedException`.
- `@PostAuthorize` runs **after** the method, so use it only for methods without side effects (reads).
- An unknown id gives **404** (a `ProblemDetail` from the controller), a foreign order **403**. The difference tells a caller which ids exist. Where that matters, answer 404 for foreign orders too.
- The rule compares the order's owner with `authentication.name`. For a client-credentials token, the name is the client id, and some identity providers let users change their display name. In production, compare with a stable, unique id (e.g. the sub claim), not with a changeable name.

3.5 A JWT Resource Server

`oauth2ResourceServer(jwt)` in the API chain accepts `Authorization: Bearer <token>`. For each request, the resource server:

1. checks the **signature** with the public key of the issuer,
2. checks the **expiry** (exp),
3. turns the scope claim into authorities `SCOPE_...`

Nothing is stored on the server: the token itself carries the identity. `TokenFlowTest` changes four characters of the signature, and the request is rejected with **401**.

Spring Security 7 also adds a **factor authority** that records how the user authenticated: `FACTOR_BEARER` for a token, `FACTOR_PASSWORD` for a password (see the `/api/me` output). Rules can require specific factors, which is the basis for multi-factor authentication.

3.6 Our Own Authorization Server

The tokens come from Spring Authorization Server, running in the same application. The client is registered in `application.yaml`:

```
security:
  oauth2:
    authorizationserver:
      client:
        bookstore-cli:                # a machine client (e.g. a script or
↪ another service)
        registration:
          client-id: bookstore-cli
          # No default: without BOOKSTORE_CLI_SECRET the application does not start
↪ (fail fast).
          # Store it hashed, e.g. BOOKSTORE_CLI_SECRET={bcrypt}$2a$10$... – the dev
↪ profile sets a demo value.
```

```

client-secret: ${BOOKSTORE_CLI_SECRET}
client-authentication-methods: client_secret_basic
authorization-grant-types: client_credentials
scopes: books.read,books.write

```

The first filter chain handles only the Authorization Server endpoints:

```

@Configuration(proxyBeanMethods = false)
public class AuthorizationServerConfiguration {

    static final String CLIENT_SECRET =
        ↪ "spring.security.oauth2.authorizationserver.client.bookstore-
        ↪ cli.registration.client-secret";

    AuthorizationServerConfiguration(Environment environment) {
        environment.getRequiredProperty(CLIENT_SECRET); // fail fast if
        ↪ BOOKSTORE_CLI_SECRET is missing
    }

    @Bean
    @Order(1)
    SecurityFilterChain authorizationServerChain(HttpSecurity http) throws Exception {
        var authorizationServer = new OAuth2AuthorizationServerConfigurer();
        http.securityMatcher(authorizationServer.getEndpointsMatcher()) // only
        ↪ these URLs
            .with(authorizationServer, Customizer.withDefaults())
            .authorizeHttpRequests(authorize ->
                ↪ authorize.anyRequest().authenticated());
        return http.build();
    }
}

```

Boot adds the rest: an RSA signing key, the public keys at `/oauth2/jwks`, the metadata at `/.well-known/oauth-authorization-server`, and a `JwtDecoder` that the resource server uses.

```

curl -u bookstore-cli:dev-only-secret -d grant_type=client_credentials -d
    ↪ scope=books.read \
    http://localhost:8080/oauth2/token

```

i Note

`client_credentials` is for machines (a script, another service). Applications with human users use the `authorization_code` grant with PKCE, where the user logs in at the Authorization Server and the application never sees the password.

! Important

Boot generates a new signing key at every start. Tokens issued before a restart become invalid. A real Authorization Server loads its key from a key store and keeps it across restarts.

3.7 OAuth2 Login (“Log in with GitHub”)

With OAuth2 login, the application is a **client** of an external provider. It is only switched on if a provider is configured (see the browser chain in section 3.1). For GitHub:

```
spring:
  security:
    oauth2:
      client:
        registration:
          github:
            client-id: ${GITHUB_CLIENT_ID}
            client-secret: ${GITHUB_CLIENT_SECRET}
```

Spring Security knows GitHub’s URLs. The login page then shows a “GitHub” link that redirects to <https://github.com/login/oauth/authorize>. `OAuth2LoginTest` checks the redirect and uses `oauth2Login()` from `spring-security-test` to simulate a returning user, without calling GitHub.

3.8 CORS

A front end on <http://localhost:5173> (e.g. a Vite dev server) may call the API. Any other origin may not:

```
@Bean
CorsConfigurationSource
↪ corsConfigurationSource(@Value("${bookstore.cors.allowed-origins}") List<String>
↪ origins) {
    var api = new CorsConfiguration();
    api.setAllowedOrigins(origins); // never "*" together with
    ↪ credentials
    api.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE"));
    api.setAllowedHeaders(List.of("Authorization", "Content-Type"));
    api.setMaxAge(3600L); // browsers may cache the
    ↪ preflight for an hour
    var source = new UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/api/**", api);
    return source;
}
```

`.cors(withDefaults())` in the API chain uses this bean. A preflight from `http://localhost:5173` gets `Access-Control-Allow-Origin`, a preflight from another origin gets **403**.

3.9 Testing

`spring-security-test` fakes the parts that are hard to set up in a test:

Helper	Simulates
<code>user("ada").roles("CUSTOMER"), @WithMockUser</code>	a logged-in user
<code>httpBasic("ada", "ada-password")</code>	real HTTP Basic against the real users
<code>formLogin().user(...).password(...)</code>	a real form login
<code>jwt().authorities(...)</code>	a request with a valid JWT
<code>oauth2Login()</code>	a user who logged in with an OAuth2 provider
<code>csrf()</code>	a valid CSRF token

`TokenFlowTest` goes further and gets a real token from the Authorization Server.

4. Common Mistakes and Best Practices

!! Caution

Never store passwords in plain text or with a fast hash (MD5, SHA-256). Use BCrypt, SCrypt or Argon2 through the `DelegatingPasswordEncoder`.

- **Do:** deny by default (`anyRequest().authenticated()`) and open up exactly what must be public.
- **Don't:** disable CSRF in a chain that uses session cookies. Only a stateless chain may do without it.
- **Do:** put ownership rules into the service with method security. A second controller or a batch job would otherwise bypass them.
- **Don't:** use `setAllowedOrigins("*")` together with credentials, and do not allow more methods and headers than needed.
- **Do:** keep access tokens short-lived and validate issuer and audience when tokens come from an external provider.
- **Don't:** commit secrets. Use environment variables or a secret store, as the `B00K-STORE_CLI_SECRET` placeholder shows.
- **Do:** deny what no rule allows (`anyRequest().denyAll()` on the API).

- **Don't:** load demo users with known passwords in every environment. Keep them in a dev-only profile and migration location.

5. Summary

- Authentication answers “who?” (401), authorization answers “what?” (403). Both are done by the filter chain before your controller runs.
- Several `SecurityFilterChains` with matchers and orders separate the API from the browser.
- Users can live in PostgreSQL. Passwords are stored as salted, slow BCrypt hashes behind an algorithm prefix.
- Browser applications need CSRF protection, stateless APIs do not. CORS controls which other origins may call the API.
- `@PreAuthorize` and `@PostAuthorize` enforce rules such as “only your own orders” on the service.
- Spring Authorization Server issues signed JWTs, and the resource server checks signature, expiry and scopes on every request.
- `spring-security-test` simulates users, tokens and logins in tests.

Next step: [Exercises](#)

6. Further Reading

- [Spring Security Reference](#) · [Servlet Architecture](#)
- [Password Storage](#) · [CSRF](#) · [CORS](#)
- [Method Security](#)
- [OAuth 2.0 Resource Server — JWT](#) · [OAuth2 Login](#)
- [Spring Authorization Server](#)
- [Spring Boot — Security](#)
- [OWASP Password Storage Cheat Sheet](#) · [Authentication Cheat Sheet](#) (brute-force protection, account logout)